b230e6.c

b230e6.c > main()

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int moisture_threshold;
    struct Node* left;
    struct Node* right;
} Node;

Node* createNode(int value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->moisture_threshold = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d%% ", root->moisture_threshold);
        inorder(root->right);
    }
}

void preorder(Node* root) {
    if (root != NULL) {
        printf("%d%% ", root->moisture_threshold);
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(Node* root) {
    if (root != NULL) {
        postorder(root->left);
```

b230e6.c

b230e6.c > main()

```c
void postorder(Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d%% ", root->moisture_threshold);
    }
}

int main() {
    Node* root = createNode(40);
    root->left = createNode(25);
    root->right = createNode(60);
    root->left->left = createNode(15);
    root->left->right = createNode(30);

    printf("=== SUGARCANE IRRIGATION ADVISORY DECISION TREE ===\n\n");

    printf("Inorder Traversal (Sorted Threshold Levels):\n");
    inorder(root);
    printf("\n\n");

    printf("Preorder Traversal (Root-First Evaluation):\n");
    preorder(root);
    printf("\n\n");

    printf("Postorder Traversal (Bottom-Up Analysis):\n");
    postorder(root);
    printf("\n");

    return 0;
}
```

```
=== SUGARCANE IRRIGATION ADVISORY DECISION TREE ===

Inorder Traversal (Sorted Threshold Levels):
15% 25% 30% 40% 60%

Preorder Traversal (Root-First Evaluation):
40% 25% 15% 30% 60%

Postorder Traversal (Bottom-Up Analysis):
15% 30% 25% 60% 40%
```